

Git & GitHub Beginner Case Study - Instructor Guide

This guide contains the expected answers and Git commands for each activity.

1. Initialize a repository

Commands:

```
mkdir StudentManagementSystem
```

```
cd StudentManagementSystem
```

```
git init
```

Answer: Initializes an empty Git repository.

2. Create project files

Create README.md, index.html, css/style.css, js/app.js.

```
git status
```

Answer: Files appear as *Untracked*.

3. Stage files

```
git add .
```

```
git status
```

Answer: Files move to the staging area and are ready to commit.

4. Commit changes

```
git commit -m "Initial project structure"
```

Answer: Meaningful commit messages make history understandable.

5. View history

```
git log
```

```
git log --oneline
```

Answer: --oneline is easier for quick review.

6. View changes

Edit README.md.

```
git diff
```

Answer: Shows modified lines before committing.

7. Create branch

```
git switch -c feature-registration (or git checkout -b feature-registration)
```

```
git branch
```

Answer: Active branch is marked with *.

8. Commit feature

Add students/student.txt.

```
git add .
```

```
git commit -m "Added registration module"
```

Answer: Branches isolate development.

9. Merge branch

```
git switch main  
git merge feature-registration  
Answer: Automatic if no conflicting changes.
```

10. Create GitHub repo

Create repository without README.
Answer: Prevents unnecessary merge conflicts with existing local repository.

11. Add remote

```
git remote add origin  
https://github.com/<username>/StudentManagementSystem.git  
git remote -v  
Answer: git remote -v verifies remotes.
```

12. Push

```
git push -u origin main  
Answer: -u sets upstream tracking.
```

13. Pull Request

```
git push origin feature-login  
Create PR on GitHub.  
Answer: Enables review before merging.
```

14. Pull latest

```
git pull origin main  
Answer: Updates local branch; reduces push conflicts.
```

15. Unstage file

```
git restore --staged temp.txt  
Answer: Removes file from staging only.
```

16. Merge conflict

Conflict markers appear in the file.
Answer: Edit the file, keep correct content, then:

```
git add .  
git commit
```

17. Graph history

```
git log --oneline --graph --all  
Answer: Displays commits, branches and merges visually.
```

18. Git vs GitHub

Answer: Git is a distributed version control system. GitHub is a cloud platform for hosting Git repositories and collaboration.

19. Commit message best practices

Answer: Use imperative tense, keep concise, describe one logical change per commit.

20. Deliverables

Answer: Repository URL, commit history, branches, PR, merge screenshot, final repository.